Design and apply context-aware access controls based on user identity, device posture, location, and application risk.

List of team members

1	Ezhilmozhi VJ	Sri Ramakrishna College of arts and science for women	srcw2322k112@srcw.ac.in
2	Dharshini M	Sri Ramakrishna College of arts and science for women	srcw2322k109@srcw.ac.in
3	Divya V	Sri Ramakrishna College of arts and science for women	srcw2322k111@srcw.ac.in
4	Harini H	Sri Ramakrishna College of arts and science for women	srcw2322k113@srcw.ac.in

Introduction

With the rapid growth of digital technologies, organizations increasingly rely on online systems, cloud platforms, and mobile devices to manage their operations. While these technologies improve efficiency and accessibility, they also introduce new security risks. Unauthorized access, data breaches, and cyber-attacks have become common challenges faced by organizations. Therefore, ensuring secure access to systems and applications has become an essential requirement.

Traditional access control systems mainly depend on static authentication methods such as usernames and passwords. However, these methods are often vulnerable to attacks such as password theft, phishing, or unauthorized device usage. As a result, modern security solutions are shifting towards more intelligent and adaptive access control models.

Context-aware access control is an advanced security approach that considers multiple contextual factors before granting access to a system or application. These factors include the identity of the user, the security posture of the device being used, the location from which the access request is made, and the risk level associated with the requested application. By evaluating these parameters, the system can determine whether the access request is legitimate and safe.

The main objective of this project is to design and implement a context-aware access control system that enhances security by analyzing user and environmental conditions in real time. The system aims to ensure that only authorized users with trusted devices and appropriate conditions are allowed to access sensitive applications.

This project demonstrates how context-based decision making can strengthen cybersecurity and help organizations protect their digital resources more effectively.

Abstract

In today's digital environment, protecting sensitive data and systems has become a major challenge. Traditional access control methods mainly rely on usernames and passwords, which are not always sufficient to prevent unauthorized access. To address this issue, context-aware access control systems are introduced. These systems make access decisions based not only on user identity but also on additional contextual factors such as device posture, user location, and application risk.

This project focuses on designing and implementing a context-aware access control mechanism that improves system security by analyzing multiple parameters before granting access. The system evaluates whether the user is authenticated, checks the security status of the device being used, identifies the geographical location of the user, and assesses the risk level of the requested application. Based on these conditions, the system decides whether to allow, restrict, or deny access.

By integrating these factors, the proposed solution enhances security, reduces unauthorized access, and ensures that only trusted users with secure devices can access critical applications. This approach supports modern security models such as Zero Trust, where every access request is continuously verified. The project demonstrates how intelligent access control can improve data protection and strengthen organizational cybersecurity.

Use Case Scenarios

Scenario 1: HR Manager Access to Employee Management System

An HR manager needs to access the internal Human Resource Management System (HRMS) to review employee records and update staff information. The user connects from the corporate office using a company-managed desktop computer. Upon launching the ZPA Client Connector, the user authenticates through Azure Active Directory using multi-factor authentication (MFA).

Zscaler Private Access evaluates the user's identity, verifies group membership (HR_Managers), and checks the device posture to confirm that the operating system is updated and endpoint protection is active. The policy engine confirms that the access request originates from a trusted office location and determines that the user is authorized.

A secure micro-tunnel is established directly to the HRMS application. The HR manager can access employee records but cannot view or connect to unrelated systems such as finance databases or development servers.

Output:

- User authenticated and authorized through Azure Active Directory with MFA
- Device posture verified as compliant
- Secure micro-tunnel created for HRMS access only
- Other internal systems remain inaccessible

Scenario 2: Developer Access to Internal Git Repository

A software developer needs to access the organization’s internal Git repository to push new code updates. The developer connects from home using a company-managed laptop. After launching the ZPA Client Connector, the user authenticates via Azure Active Directory with MFA.

Zscaler Private Access evaluates the user's identity and confirms membership in the Developers_Team group. The system checks the device posture to ensure antivirus protection is active and the operating system has the latest security patches installed. The connection location is verified as an approved region.

Once the policy engine confirms that all conditions are satisfied, a secure micro-tunnel is established to the Git repository server. The developer can clone, pull, and push code but cannot access internal finance systems or HR databases.

Output:

- User authenticated and verified through Azure Active Directory with MFA
- Device security posture validated
- Secure micro-tunnel established for Git repository access
- Access to other internal systems blocked

Scenario 3: Executive Access to Business Intelligence Dashboard

A company executive needs to access the corporate business intelligence dashboard to review company performance metrics. The executive connects from a company-issued tablet while traveling.

After launching the ZPA Client Connector, the user authenticates via Azure Active Directory using MFA. Zscaler Private Access evaluates the user's executive group membership and checks the device posture to ensure the tablet is registered and compliant with the organization's security policies.

Because the login originates from a foreign location and the dashboard contains sensitive data, the policy engine assigns a moderate risk score. Access is granted with restrictions such as disabled downloads and monitored session activity.

Output:

- Executive user authenticated using MFA
- Device posture verified as compliant
- Access granted with restricted permissions due to location risk
- Session activity monitored for unusual behavior

Scenario 4: IT Support Technician Access to Internal Support Tools

An IT support technician needs to access internal support tools to assist an employee with a system issue. The technician connects from home using a company-managed workstation.

After launching the ZPA Client Connector, the technician authenticates through Azure Active Directory with MFA. Zscaler Private Access evaluates the user's identity, confirms membership in the IT_Support group, and verifies device posture compliance including firewall status and endpoint protection.

The policy engine confirms that the connection originates from an approved home-office network and grants secure micro-tunnel access to the Service Desk system and the remote desktop jump host used for technical support.

Output:

- IT technician authenticated and verified through MFA
- Device posture confirmed as compliant
- Secure micro-tunnels established for support tools
- Access to unrelated applications restricted

Scenario 5: Marketing Employee Access to CRM System from Public Network

A marketing employee needs to update campaign data in the corporate CRM system. The employee connects from a coffee shop using a company-managed smartphone.

After launching the ZPA Client Connector, the user authenticates through Azure Active Directory with MFA. Zscaler Private Access evaluates the user's identity, confirms membership in the Marketing_CRM_Users group, and checks the device posture to ensure it is managed and compliant.

However, the system identifies that the connection is coming from a public Wi-Fi network, which increases the security risk. Instead of granting direct access, the policy engine routes the session through a secure browser isolation environment to prevent data leakage.

Output:

• User authenticated and device verified as compliant
• Network location identified as high risk
• Access provided through browser isolation
• Sensitive data protected from exposure on public Wi-Fi

Stage – 1

Title

Design and Apply Context-Aware Access Controls Based on User Identity, Device Posture, Location, and Application Risk

Overview

With the rapid evolution of cyber threats and the decline of the traditional network perimeter, modern organizations are moving away from conventional VPN-based remote access solutions. Traditional VPN models often provide users with broad network-level access once they are authenticated. This approach increases the attack surface and creates opportunities for attackers to move laterally within the network if a single account or device becomes compromised.

To overcome these limitations, organizations are increasingly adopting the **Zero Trust security architecture**, which is based on the principle of **"never trust, always verify."** In this model, trust is not automatically granted based on a user's location within the network. Instead, every access request is continuously verified using multiple contextual factors such as user identity, device security posture, geographical location, and the risk level of the requested application.

This project focuses on implementing a context-aware access control system using **Zscaler Private Access (ZPA)**, a cloud-delivered Zero Trust Access solution. ZPA enables secure, identity-based access to internal applications without exposing the corporate network to the internet. Unlike traditional VPNs that connect users directly to the internal network, ZPA follows a service-initiated model that establishes secure, one-to-one connections between authenticated users and specific applications.

Through this approach, users can access only the applications they are authorized to use, based on their identity and contextual security conditions. The internal network and IP addresses remain completely hidden from external users, making the applications invisible to unauthorized individuals and significantly reducing the risk of cyberattacks such as port scanning and distributed denial-of-service (DDoS) attacks.

The project ultimately aims to implement a **least-privilege access model**, where users and devices are granted only the minimum level of access required to perform their tasks. By separating application access from network access, the system prevents lateral movement across the network and ensures that users cannot discover or connect to other internal systems. This strategy significantly strengthens organizational security, reduces the overall attack surface, and improves both operational efficiency and user experience in modern access management environments.

List of team members

S NO	Name	College Name	Contact
1	Ezhilmozhi VJ	Sri Ramakrishna College of arts and science for women	srcw2322k112@srcw.ac.in
2	Dharshini M	Sri Ramakrishna College of arts and science for women	srcw2322k109@srcw.ac.in
3	Divya V	Sri Ramakrishna College of arts and science for women	srcw2322k111@srcw.ac.in
4	Harini H	Sri Ramakrishna College of arts and science for women	srcw2322k113@srcw.ac.in

List of Vulnerability:

S NO	Category Name	Vulnerability Name	CWE No
1	Broken Access Control	Absolute Path Traversal	CWE-36
2	Security Misconfiguration	ASP.NET Misconfiguration: Password in Configuration File	CWE-13
3	Software Supply Chain Failures	Use of Unmaintained Third Party Components	CWE-1104
4	Cryptographic Failures	Cleartext Transmission of Sensitive Information	CWE-319
5	Injection	Improper Neutralization of Equivalent Special Elements	CWE-76
6	Insecure Design	Plaintext Storage of a Password	CWE-256
7	Authentication Failures	Improper Authentication	CWE-287
8	Software or Data Integrity Failures	Untrusted Search Path	CWE-426
9	Security Logging & Alerting Failures	Omission of Security-relevant Information	CWE-223
10	Mishandling of Exceptional Conditions	Failure to Handle Missing Parameter	CWE-234

REPORT:

1.Vulnerability Name: Broken Access Control

CWE/CVE : CWE-36

Owasp/Sans category: A01:2025 Broken Access Control

Description

Broken Access Control is a security vulnerability where a system fails to properly enforce restrictions on authenticated users, allowing them to perform actions or access data beyond their authorized permissions. Access control mechanisms are designed to ensure that users can only access resources that correspond to their roles and privileges. When these mechanisms are incorrectly implemented or missing, attackers can exploit the weakness to gain unauthorized access.

This vulnerability commonly occurs when applications rely only on client-side validation or when authorization checks are not consistently applied on the server side. As a result, users may be able to access restricted pages, modify sensitive data, or perform administrative actions without proper permission.

Typical examples of broken access control include privilege escalation, where a normal user gains administrative rights; insecure direct object references (IDOR), where users manipulate object identifiers such as user IDs in URLs to access other users' data; and bypassing authentication checks by directly accessing restricted endpoints. Because access control flaws are often easy to exploit and difficult to detect without proper security testing, they remain one of the most serious risks in modern web applications.

Business Impact

Broken Access Control can have severe consequences for organizations. One of the primary risks is **unauthorized exposure of sensitive information**, such as personal user data, financial records, or confidential business information. This can result in significant data breaches that compromise customer trust and damage the organization's reputation.

Another major impact is **financial loss**, which may occur due to fraud, unauthorized transactions, or the cost of responding to security incidents. Organizations may also face **legal and regulatory penalties** if the breach violates data protection regulations and compliance requirements.

Operational disruptions are also possible. Attackers who gain unauthorized privileges may modify or delete critical data, interfere with application functionality, or take control of administrative systems. Such actions can interrupt business operations, affect service availability, and lead to long recovery periods.

Overall, broken access control not only threatens the security of applications but also affects the organization's credibility, financial stability, and compliance obligations.

Recommendations

To mitigate the risks associated with broken access control, organizations must implement strong and well-structured authorization mechanisms throughout their applications. One effective approach is to adopt **Role-Based Access Control (RBAC)**, where user permissions are defined according to their roles within the system.

Access control checks should always be enforced on the **server side**, ensuring that authorization is verified for every request regardless of client-side controls. Systems should follow a **deny-by-default principle**, meaning that access is restricted unless explicitly granted.

Developers should also validate user permissions for every action or request, particularly when accessing sensitive resources or performing administrative tasks. Implementing **secure session management** practices, such as protecting session tokens and preventing session hijacking, is also essential.

Regular **security testing**, including vulnerability assessments, penetration testing, and code reviews, can help identify access control weaknesses early in the development process. Additionally, organizations should maintain proper **logging and monitoring mechanisms** to detect unauthorized access attempts and respond quickly to potential attacks.

By applying these practices, organizations can significantly reduce the risk of unauthorized access and protect both user data and business systems.

2. Vulnerability Name: Security Misconfiguration

CWE/CVE : CWE-13

Owasp/Sans category: A02:2025 Security Misconfiguration

Description

Security Misconfiguration occurs when security settings in an application, server, database, or cloud service are improperly configured or left at their default values. This vulnerability arises when systems are deployed without proper security hardening, leaving them exposed to potential attacks.

In many cases, applications include unnecessary features, open ports, default accounts, or overly permissive permissions that attackers can exploit. Security misconfiguration may also occur when error messages reveal sensitive information about the system architecture, software versions, or database structure. If these details are exposed, attackers can use them to identify and exploit known vulnerabilities.

Another common cause of security misconfiguration is failing to update or patch software components regularly. Outdated frameworks, libraries, and operating systems may contain known security flaws that attackers can easily exploit. Similarly, enabling debugging features or leaving administrative interfaces accessible on the internet can significantly increase the risk of unauthorized access.

Because modern applications rely on multiple components such as web servers, application servers, databases, APIs, and cloud services, security misconfiguration can occur at many levels. Without proper configuration management and security controls, these weaknesses can provide attackers with entry points to compromise the system.

Business Impact

Security misconfiguration can lead to serious consequences for organizations. One of the primary impacts is **unauthorized access to sensitive systems or data**, which may result in data breaches involving confidential information such as user credentials, financial records, or internal company data.

Another major impact is **system compromise**. Attackers may exploit misconfigured services or default credentials to gain control over servers or applications. This can allow them to modify system settings, install malicious software, or disrupt normal operations.

Organizations may also experience **financial losses** due to incident response costs, operational downtime, and potential regulatory penalties. Data breaches caused by misconfiguration can lead to legal consequences if they violate data protection regulations.

Additionally, **reputational damage** is a significant concern. Customers and stakeholders may lose trust in an organization if its systems are compromised due to poor security practices.

This can affect long-term business relationships and the organization's credibility in the market.

Recommendations

To reduce the risk of security misconfiguration, organizations should implement a strong and consistent **secure configuration management process** across all systems and environments.

First, all systems should be properly **hardened before deployment**. This includes disabling unnecessary services, closing unused ports, and removing default accounts or credentials. Only essential features required for the application should remain enabled.

Second, organizations should ensure that **software components, frameworks, and operating systems are regularly updated and patched** to address known vulnerabilities. Automated patch management systems can help maintain up-to-date environments.

It is also important to **restrict access to administrative interfaces** and sensitive configuration settings. These interfaces should be protected using strong authentication and should not be publicly accessible.

Proper **error handling mechanisms** should be implemented so that applications do not expose sensitive technical details in error messages. Instead, generic messages should be displayed to users while detailed logs are securely stored for administrators.

Finally, organizations should conduct **regular security testing and configuration reviews**, including vulnerability scanning and security audits. Continuous monitoring and logging can help detect configuration issues early and prevent them from being exploited.

By following these practices, organizations can significantly reduce the risk of security misconfiguration and strengthen the overall security posture of their applications and infrastructure

Threat agents/attack vectors	Security weakness	Impacts
The threat vectors can vary quite a lot here, it can range from not encrypting traffic between servers to having a shadow API on the network that is not maintained and poorly configured to allow for SSH access.	This vulnerability can occur on any level that requires configuration, ranging from the network to the application level. This vulnerability can be tested manually but if you have access to the source code and even if you do not it is certainly recommended to run automated tools as they can reliably identify and report on known issues.	This vulnerability might lead to an attacker being able to fully take over all the infrastructure even so this is an issue that should be handled with great care.

3. Vulnerability Name: Software Supply Chain Failures

CWE/CVE : CWE-1104

Owasp/Sans category: A03:2025 Software Supply Chain Failures

Description

Software Supply Chain Failures refer to security risks that arise from vulnerabilities or compromises in third-party components, libraries, tools, or services used during software development and deployment. Modern applications rely heavily on external dependencies such as open-source libraries, package managers, APIs, cloud services, and development tools. When these components are not properly verified, monitored, or updated, they can introduce security weaknesses into the application.

Attackers may exploit vulnerabilities in third-party software or inject malicious code into dependencies to compromise the entire system. For example, if a developer includes a library from a public repository that contains malicious code or an unpatched vulnerability, the application automatically inherits that risk. Similarly, compromised build pipelines, package repositories, or development tools can allow attackers to distribute malicious updates to many applications simultaneously.

Another common issue occurs when organizations do not maintain a clear inventory of the components used in their software. Without proper tracking and visibility, it becomes difficult to identify vulnerable or outdated dependencies. This lack of control over the software supply chain can significantly increase the risk of security incidents.

Overall, software supply chain failures highlight the importance of securing not only the application code but also every external component involved in the software development lifecycle.

Business Impact

Software supply chain failures can have serious consequences for organizations. One of the most significant impacts is the **introduction of hidden vulnerabilities** into applications through third-party components. Since these components are often widely used, a single vulnerability can affect multiple systems simultaneously.

Another major impact is **large-scale security breaches**. If attackers successfully compromise a widely used dependency or development tool, they can distribute malicious code to many organizations at once. This can lead to widespread attacks affecting thousands of applications and users.

Organizations may also experience **financial losses** due to incident response costs, system recovery, and potential legal penalties. If customer data is compromised as a result of a vulnerable dependency, the organization may face regulatory consequences and compensation claims.

In addition, **reputational damage** can occur when customers and partners lose trust in the organization's ability to secure its software products. A compromised supply chain can undermine confidence in the company's development practices and reduce market credibility.

Recommendations

To prevent software supply chain failures, organizations should implement strong security practices throughout the software development lifecycle.

First, organizations should maintain a **Software Bill of Materials (SBOM)** that lists all third-party libraries, frameworks, and dependencies used in the application. This helps developers track components and quickly identify vulnerable packages.

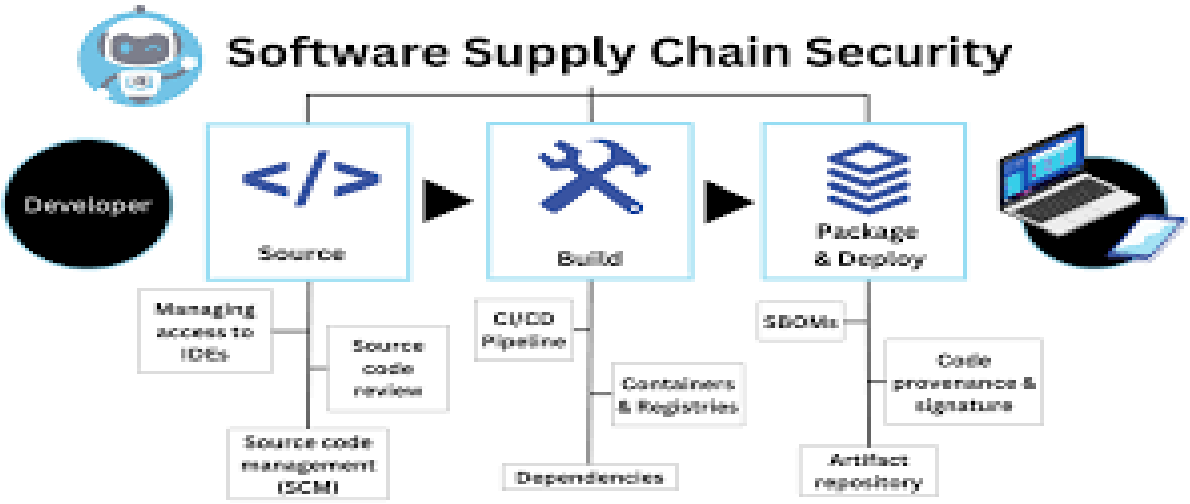
Second, all external components should be **downloaded only from trusted and verified sources**. Developers should avoid using unverified or unofficial libraries and ensure that dependencies come from reputable repositories.

Third, organizations should regularly **scan dependencies for known vulnerabilities** using automated security tools. Dependency scanning and vulnerability management tools can help identify outdated or insecure components before they are deployed.

Another important practice is to **secure the build and deployment pipeline**. Access to build servers, package repositories, and continuous integration systems should be restricted and monitored to prevent unauthorized modifications.

Finally, organizations should implement **regular updates and patch management** for third-party components. Keeping dependencies updated reduces the risk of known vulnerabilities being exploited.

By maintaining strong control and visibility over third-party components and development tools, organizations can significantly reduce the risks associated with software supply chain



4.Vulnerability Name: Cryptographic Failures

CWE/CVE : CWE-139

Owasp/Sans category: A04:2025 Cryptographic Failures

Description

Cryptographic Failures refer to weaknesses that occur when sensitive data is not properly protected through encryption or when cryptographic mechanisms are incorrectly implemented. Cryptography is used to secure data by converting it into an unreadable format so that only authorized users with the correct key can access the information. When encryption is missing, weak, or improperly configured, attackers may be able to intercept or access confidential data.

These failures commonly occur when applications store sensitive information such as passwords, credit card numbers, or personal data without encryption. Another common issue is the use of outdated or weak cryptographic algorithms that can be easily broken by attackers. Improper key management, such as hard-coding encryption keys in source code or using the same key for long periods, can also lead to security vulnerabilities.

Cryptographic failures may also arise when secure communication protocols are not used. For example, transmitting sensitive data over unencrypted connections can allow attackers to intercept the information through techniques such as man-in-the-middle attacks. Because cryptography is a critical component in protecting data confidentiality and integrity, failures in its implementation can expose sensitive information to unauthorized parties.

Business Impact

Cryptographic failures can result in serious consequences for organizations. One of the most significant impacts is the **exposure of sensitive data**, including personal information, login credentials, financial records, and confidential business data. If this information is accessed by unauthorized individuals, it can lead to data breaches affecting both the organization and its users.

Another major impact is **financial loss**. Organizations may face costs related to incident response, compensation for affected customers, and regulatory penalties if data protection laws are violated. Many industries require strict security standards for protecting sensitive data, and failure to comply can result in legal consequences.

Cryptographic failures can also cause **reputational damage**. When customers discover that their data has been exposed due to weak encryption practices, they may lose trust in the organization’s ability to protect their information. This loss of trust can negatively affect customer relationships and long-term business success.

In addition, attackers who gain access to sensitive data may use it for **identity theft, fraud, or further cyberattacks**, increasing the overall risk to the organization and its users.

Recommendations

To prevent cryptographic failures, organizations should implement strong and properly configured cryptographic practices. Sensitive data should always be **encrypted both at rest and in transit** using secure and modern encryption algorithms.

Organizations should avoid using outdated or insecure cryptographic methods and instead rely on **trusted and well-established cryptographic libraries and protocols**. Developers should not attempt to create custom encryption algorithms, as these are often insecure.

Proper **key management practices** should also be implemented. Encryption keys should be securely generated, stored, and rotated regularly. Keys should never be hard-coded in application source code or stored in publicly accessible locations.

It is also important to use **secure communication protocols**, such as HTTPS, to protect data transmitted between clients and servers. Regular security testing, vulnerability scanning, and code reviews can help identify weaknesses in cryptographic implementations.

By following these best practices, organizations can ensure that sensitive data remains protected and reduce the risk of cryptographic failures within their applications.

5.Vulnerability Name: Injection

CWE/CVE : CWE-76

Owasp/ Sans category: A05:2025 Injection

Description

Injection is a security vulnerability that occurs when an application accepts untrusted input and sends it to an interpreter or database without proper validation or sanitization. This allows attackers to insert malicious code or commands into the input fields, which the system

then executes as part of a query or command. As a result, attackers may manipulate the application's behavior or gain unauthorized access to sensitive information.

Injection vulnerabilities can occur in many forms, including SQL injection, command injection, and code injection. One of the most common types is **SQL injection**, where attackers insert malicious SQL queries into input fields such as login forms or search boxes. If the application directly includes this input in a database query without proper validation, the attacker may retrieve, modify, or delete database records.

Injection vulnerabilities usually occur due to insufficient input validation, lack of parameterized queries, or improper handling of user inputs. Because web applications frequently accept user input through forms, URLs, and API requests, they become vulnerable if the input is not carefully checked and processed.

Business Impact

Injection vulnerabilities can have severe consequences for organizations. One of the most critical impacts is **unauthorized access to sensitive data** stored in databases. Attackers may retrieve confidential information such as usernames, passwords, financial records, or personal details.

Another major impact is **data manipulation or deletion**. Attackers may alter database records, insert false data, or delete important information, leading to system instability and loss of data integrity.

Injection attacks can also lead to **complete system compromise** if attackers gain administrative privileges or execute commands on the server. This may allow them to install malware, modify application functionality, or disrupt services.

In addition, organizations may face **financial losses, legal penalties, and reputational damage** if a successful injection attack results in data breaches or service interruptions. Customers may lose trust in the organization's ability to protect their information.

Recommendations

To prevent injection vulnerabilities, organizations should implement strong input validation and secure coding practices. All user inputs should be **validated and sanitized** before being processed by the application.

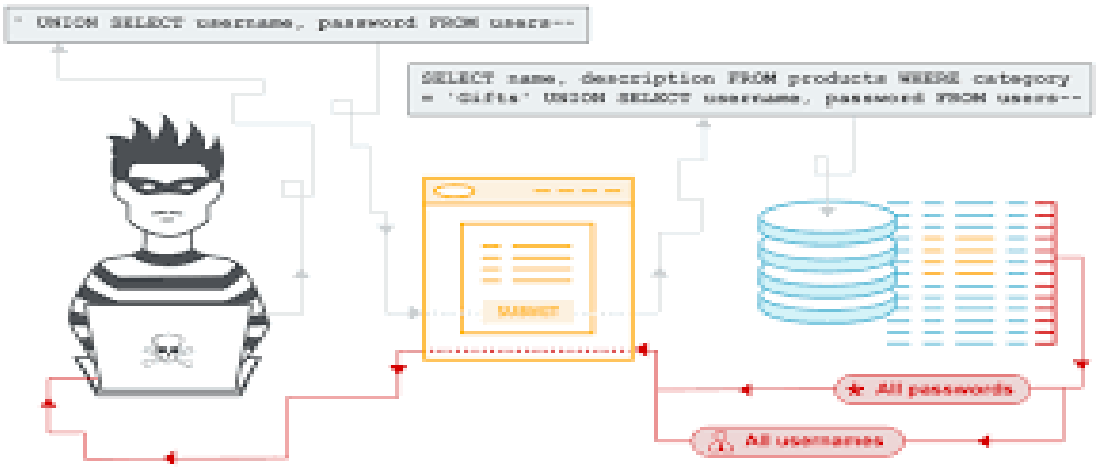
Developers should use **parameterized queries or prepared statements** instead of directly embedding user input into database queries. This ensures that user input is treated as data rather than executable code.

Applications should also implement **proper input validation mechanisms** that allow only expected and safe data formats. Using frameworks and libraries that provide built-in protection against injection attacks can further reduce the risk.

Another important practice is to **apply the principle of least privilege** to database accounts. Applications should use database credentials with limited permissions so that even if an injection attack occurs, the damage is minimized.

Regular **security testing, code reviews, and vulnerability scanning** should be conducted to identify and fix injection weaknesses early. Logging and monitoring systems should also be used to detect suspicious input patterns or abnormal database activity.

By following these security measures, organizations can effectively reduce the risk of injection attacks and protect their applications and data from unauthorized access.



6.Vulnerability Name: Insecure Design

CWE/CVE : CWE-256

Owasp/ Sans category: A06:2025 Insecure Design

Description

Insecure Design refers to security weaknesses that arise due to poor design decisions or the absence of proper security controls during the application design phase. Unlike implementation vulnerabilities caused by coding errors, insecure design occurs when security is not considered as a fundamental part of the system architecture. As a result, even if the application is coded correctly, it may still be vulnerable to attacks because the underlying design does not adequately protect against potential threats.

This vulnerability often occurs when developers focus primarily on functionality and performance while neglecting security requirements. For example, an application may not implement proper authentication mechanisms, fail to limit the number of login attempts, or allow unrestricted access to sensitive features. These design flaws can create opportunities for attackers to exploit the system.

Insecure design can also result from the absence of threat modeling, risk analysis, and secure design principles during the development lifecycle. Without proper planning and security evaluation, applications may lack critical safeguards needed to protect data and system resources.

Business Impact

Insecure design can have serious consequences for organizations because the vulnerabilities exist at the architectural level of the application. One major impact is **increased exposure to security attacks**, as attackers can exploit design flaws to bypass security mechanisms or gain unauthorized access to system resources.

Another significant impact is **data compromise**, where sensitive information such as user credentials, personal details, or financial data may be exposed due to weak system design. Since the vulnerability is embedded in the application structure, it can affect multiple components and lead to widespread security issues.

Organizations may also face **high remediation costs** because fixing design-level vulnerabilities often requires major changes to the system architecture. This can involve redesigning components, rewriting parts of the application, or redeploying systems.

Additionally, insecure design can lead to **reputational damage and loss of customer trust** if security incidents occur. Businesses that fail to incorporate security into their design process may struggle to maintain confidence among users, partners, and stakeholders.

Recommendations

To prevent insecure design vulnerabilities, organizations should adopt a **secure development lifecycle (SDLC)** that integrates security considerations at every stage of the software development process.

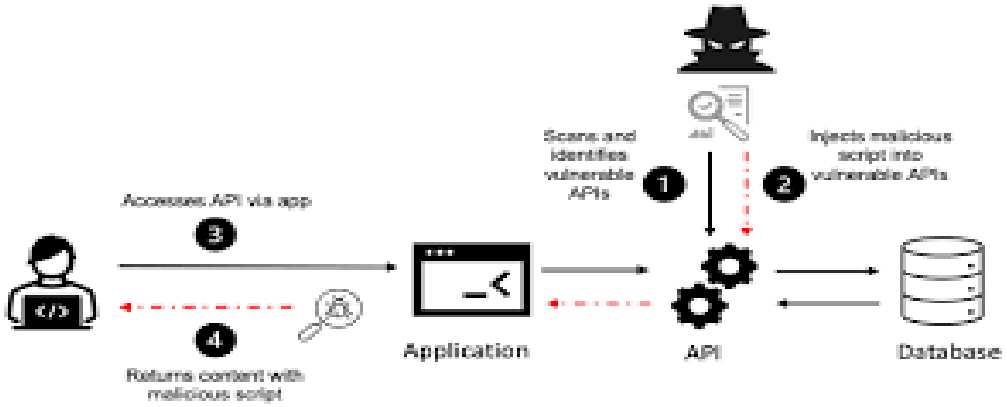
One important practice is **threat modeling**, which helps developers identify potential threats and design appropriate security controls before the system is built. By analyzing possible attack scenarios early, teams can design systems that are resilient to common security threats.

Organizations should also implement **secure design principles**, such as least privilege, defense in depth, and fail-safe defaults. These principles ensure that systems are designed to minimize risk and limit the impact of potential attacks.

Another recommendation is to establish **security requirements and design reviews** during the planning and architecture phases. Security experts should evaluate system designs to identify weaknesses and suggest improvements before development begins.

Regular **security training for developers and architects** is also essential to ensure they understand secure design practices and can apply them effectively in their projects.

By incorporating security into the design phase and following secure architecture principles, organizations can significantly reduce the risk of insecure design vulnerabilities and build more secure applications.



7.Vulnerability Name: Authentication Failures

CWE/CVE : CWE-287

Owasp/ Sans category: A07:2025 Authentication Failures

Description

Authentication Failures occur when an application does not properly verify the identity of users before granting access to system resources. Authentication is the process of confirming that a user is who they claim to be, typically through credentials such as usernames, passwords, or other verification methods. When authentication mechanisms are weak or improperly implemented, attackers may be able to bypass login systems and gain unauthorized access to accounts or sensitive information.

These failures can occur due to several reasons, such as weak password policies, lack of account lockout mechanisms, insecure session management, or improper implementation of authentication protocols. For example, if an application allows unlimited login attempts, attackers may perform **brute-force attacks** to guess user passwords. Similarly, storing passwords in plain text or using weak hashing methods can make it easier for attackers to obtain user credentials.

Authentication failures may also occur when session tokens are not securely managed. If session identifiers are predictable, exposed in URLs, or not properly invalidated after logout, attackers may hijack active sessions and impersonate legitimate users. Because authentication is a fundamental security control in web applications, weaknesses in this area can lead to serious security risks.

Business Impact

Authentication failures can lead to significant consequences for organizations. One of the most serious impacts is **unauthorized account access**, where attackers gain control over user accounts. This may allow them to access personal information, perform unauthorized transactions, or misuse system resources.

Another major impact is **data breaches**. If attackers successfully bypass authentication controls, they may access sensitive data stored within the application, including confidential business information and customer records. This exposure can result in regulatory violations and legal penalties.

Organizations may also experience **financial losses** due to fraudulent activities performed through compromised accounts. Additionally, security incidents caused by authentication failures can damage the organization's reputation and reduce customer trust.

In severe cases, attackers may gain access to **administrator accounts**, allowing them to control system settings, modify application functionality, or disrupt services. This can lead to operational downtime and significant recovery costs.

Recommendations

To prevent authentication failures, organizations should implement strong and secure authentication mechanisms. One important measure is enforcing **strong password policies**, requiring users to create complex passwords and update them periodically.

Applications should also implement **multi-factor authentication (MFA)**, which requires users to provide additional verification factors such as a one-time password or biometric authentication. This significantly reduces the risk of unauthorized access even if passwords are compromised.

Another recommendation is to implement **account lockout mechanisms** after a certain number of failed login attempts to prevent brute-force attacks. Proper **session management practices** should also be followed, including generating secure session tokens, protecting them from exposure, and invalidating them after logout or inactivity.

Passwords should always be **securely stored using strong hashing algorithms** and never stored in plain text. Additionally, organizations should conduct regular **security testing and monitoring** to detect suspicious login attempts and unauthorized access activities.

By implementing these security practices, organizations can strengthen their authentication mechanisms and protect their applications from unauthorized access and account compromise.

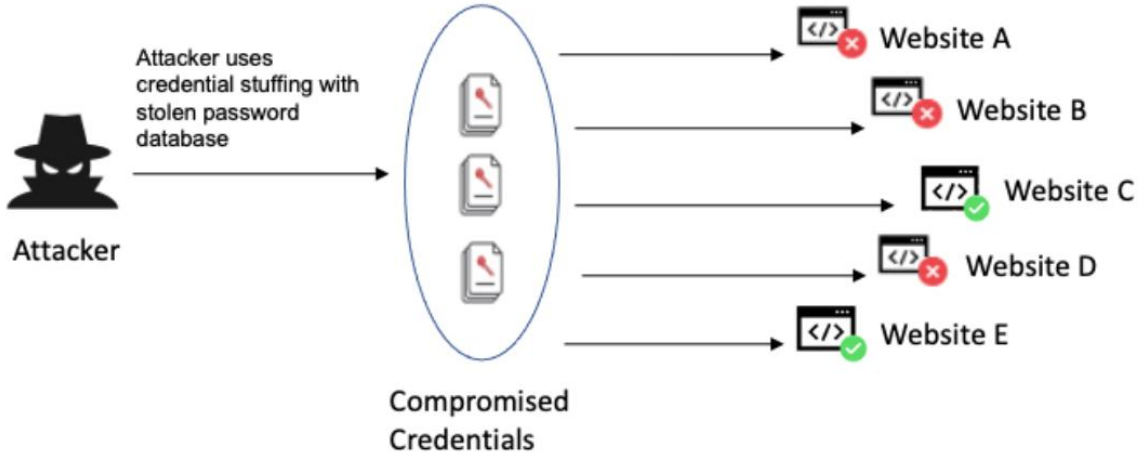


Figure: Identification and authentication failures attack scenario

8.Vulnerability Name: Software or Data Integrity Failures

CWE/CVE : CWE-426

Owasp/ Sans category: A08:2025 Software or Data Integrity Failures

Description

Software or Data Integrity Failures occur when an application fails to ensure that software updates, critical data, or system processes are protected from unauthorized modification. Integrity refers to the assurance that data and software remain accurate, consistent, and unaltered unless properly authorized. When integrity controls are missing or weak, attackers may modify application code, configuration files, or stored data.

These failures often arise when applications download or update software from untrusted sources without verifying its authenticity. For example, if an application automatically installs updates without validating their digital signatures, attackers may insert malicious code into the update process. Similarly, insecure deserialization can allow attackers to manipulate serialized data and execute harmful code within the system.

Another common cause of integrity failures is the lack of proper verification mechanisms for data and software components. If applications do not validate the integrity of files, packages, or system inputs, attackers may alter them to introduce vulnerabilities or malicious behavior. Because many modern systems rely on automated deployment pipelines and external components, maintaining data and software integrity is critical to preventing security breaches.

Business Impact

Software or data integrity failures can have serious consequences for organizations. One of the major impacts is **unauthorized modification of application code or data**, which can

compromise the reliability and security of the system. Attackers may insert malicious code that allows them to control the application or steal sensitive information.

Another significant impact is **system compromise**. If attackers successfully alter software components or update mechanisms, they may distribute malware or backdoors that affect multiple users or systems. This can lead to widespread security incidents and large-scale attacks.

Organizations may also experience **financial losses and operational disruptions** as a result of corrupted data or compromised software. Restoring systems and recovering accurate data can require significant time and resources.

Additionally, **loss of trust and reputational damage** may occur if customers or partners discover that the organization's software or data has been tampered with. This can reduce confidence in the organization's products and services and negatively affect long-term business relationships.

Recommendations

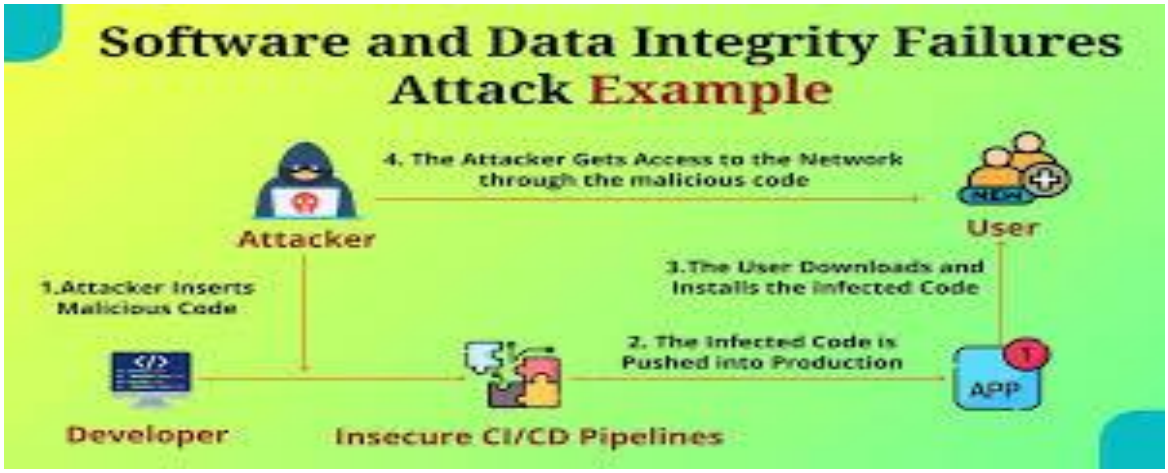
To prevent software or data integrity failures, organizations should implement strong mechanisms to verify the authenticity and integrity of software and data. One important practice is the use of **digital signatures and cryptographic verification** for software updates and packages. This ensures that updates come from trusted sources and have not been modified.

Organizations should also implement **secure update mechanisms** that verify the integrity of files before installation. Automated deployment pipelines should include integrity checks to ensure that only verified code is deployed to production environments.

Another important measure is to protect **critical data and configuration files** using access control mechanisms and integrity monitoring tools. These tools can detect unauthorized changes and alert administrators immediately.

Developers should also avoid insecure deserialization and ensure that **data inputs are properly validated and verified** before being processed by the application.

Finally, organizations should conduct **regular security audits, monitoring, and logging** to detect any unauthorized modifications to software or data. By implementing these practices, organizations can maintain the integrity of their systems and reduce the risk of malicious tampering.



9.Vulnerability Name: Security Logging & Alerting Failures

CWE/CVE : CWE-223

Owasp/ Sans category: A09:2025 Security Logging & Alerting Failures

Description

Security Logging and Alerting Failures occur when an application does not properly record security-related events or fails to generate alerts when suspicious activities occur. Logging and monitoring mechanisms are essential for detecting, analyzing, and responding to security incidents. When these mechanisms are missing, poorly implemented, or not actively monitored, attackers can perform malicious activities without being detected.

This vulnerability often arises when important events such as login attempts, access to sensitive data, system errors, or configuration changes are not logged. Even if logs are generated, organizations may fail to review them regularly or may not have automated systems that trigger alerts for unusual behavior.

Another common issue is storing logs in insecure locations where they can be modified or deleted by attackers. If attackers gain access to the system and alter or remove logs, it becomes difficult for administrators to investigate incidents or determine how the attack occurred.

Without proper logging and alerting, organizations may remain unaware of security breaches for long periods. This delay in detection allows attackers to continue exploiting the system and potentially cause more damage.

Business Impact

Security logging and alerting failures can significantly impact an organization’s ability to detect and respond to cyber threats. One major consequence is **delayed detection of security**

breaches. If suspicious activities are not logged or monitored, attackers may remain undetected while they steal data or compromise systems.

Another serious impact is **difficulty in incident investigation and response**. Without accurate logs, security teams cannot analyze what happened, identify the source of the attack, or determine the extent of the damage. This makes it harder to recover from the incident and prevent similar attacks in the future.

Organizations may also face **financial losses and legal consequences** if data breaches occur and they are unable to provide evidence of proper monitoring and response measures. Many regulatory standards require organizations to maintain detailed logs and monitoring systems.

Additionally, the absence of effective logging and alerting can lead to **reputational damage**, as customers and partners may lose confidence in the organization's ability to detect and manage security incidents.

Recommendations

To mitigate security logging and alerting failures, organizations should implement comprehensive logging and monitoring mechanisms across all systems and applications. Important security events such as login attempts, authentication failures, access to sensitive resources, and system configuration changes should be properly recorded.

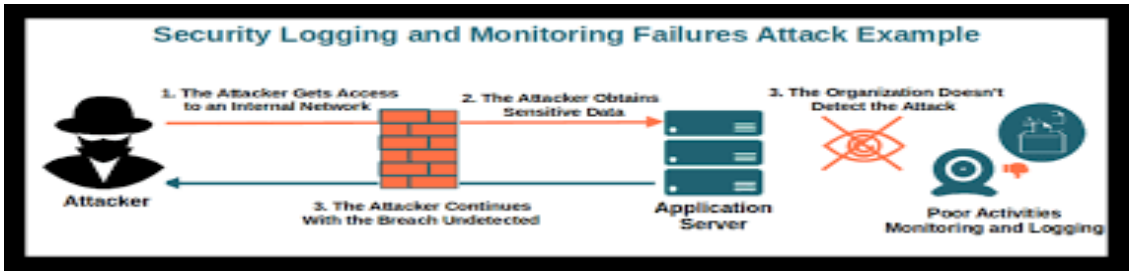
Logs should be **stored securely** and protected from unauthorized access or modification. It is recommended to use centralized logging systems that collect logs from multiple sources and store them in a secure environment.

Organizations should also implement **real-time monitoring and alerting systems** that automatically detect suspicious activities and notify administrators immediately. This enables security teams to respond quickly and reduce the impact of potential attacks.

Another important practice is to maintain **regular log reviews and analysis** to identify unusual patterns or potential threats. Automated security tools can assist in analyzing large volumes of log data and detecting anomalies.

Finally, organizations should ensure that logging and monitoring systems are integrated into their **incident response processes**, allowing security teams to quickly investigate and resolve security incidents.

By implementing strong logging, monitoring, and alerting practices, organizations can improve their ability to detect attacks early and respond effectively to security threats.



10.Vulnerability Name: Mishandling of Exceptional Conditions

CWE/CVE : CWE-234

Owasp/ Sans category: A10:2025 Mishandling of Exceptional Conditions

Description

Mishandling of Exceptional Conditions occurs when an application does not properly handle unexpected situations, errors, or abnormal system states. Exceptional conditions may include invalid user inputs, system failures, resource unavailability, or unexpected runtime errors. If these conditions are not managed correctly, they may expose sensitive information, cause system crashes, or create security vulnerabilities.

In many cases, applications display detailed error messages that reveal internal system information such as database queries, file paths, server configurations, or software versions. Attackers can use this information to understand the system architecture and identify potential weaknesses to exploit.

Another common issue occurs when applications fail to properly validate inputs or handle exceptions in a secure way. For example, improper error handling may allow attackers to bypass security checks, disrupt system operations, or trigger denial-of-service conditions. Poor exception management may also lead to inconsistent application behavior, which attackers can exploit to gain unauthorized access.

Proper handling of exceptional conditions is essential to maintain system stability, protect sensitive information, and ensure that errors do not lead to security vulnerabilities.

Business Impact

Mishandling exceptional conditions can have several negative impacts on an organization. One of the major consequences is **information disclosure**, where detailed error messages reveal sensitive technical information that can assist attackers in launching further attacks.

Another significant impact is **system instability or service disruption**. If exceptions are not handled correctly, applications may crash or behave unpredictably, leading to downtime and reduced service availability for users.

Organizations may also experience **security vulnerabilities** if attackers exploit improperly handled exceptions to bypass validation checks or manipulate application behavior. This can potentially lead to unauthorized access to system resources or sensitive data.

In addition, repeated application failures or exposed system errors can cause **reputational damage**, as users may lose confidence in the reliability and security of the system.

Recommendations

To prevent mishandling of exceptional conditions, organizations should implement proper **exception handling mechanisms** throughout their applications. Developers should anticipate possible error scenarios and design the application to handle them safely without exposing sensitive information.

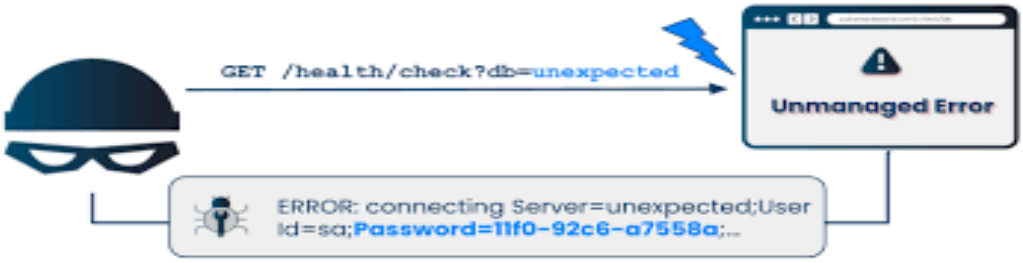
Applications should display **generic error messages** to users while logging detailed error information securely on the server for administrators and developers. This prevents attackers from gaining insight into the system's internal structure.

Input validation should also be implemented to ensure that unexpected or malicious inputs do not trigger unhandled exceptions. Proper validation and sanitization of user inputs can help prevent many runtime errors.

Another important practice is to implement **secure logging and monitoring** to track errors and identify potential issues early. Regular testing, including stress testing and security testing, can help identify exception-handling weaknesses before the application is deployed.

Finally, developers should follow **secure coding practices and frameworks** that provide built-in mechanisms for managing exceptions and maintaining application stability.

By properly managing exceptional conditions, organizations can improve system reliability, prevent information disclosure, and reduce the risk of security vulnerabilities.



Stage – 2

Overview

A Nessus scan is a vulnerability assessment process used to identify security weaknesses in computer systems and networks. It helps organizations detect vulnerabilities, misconfigurations, and potential threats before they can be exploited by attackers. The typical workflow of a Nessus scan involves several important stages, from preparation to remediation and continuous monitoring.

1. Preparation

The first step in performing a Nessus scan is defining the objectives of the assessment. This includes identifying the systems, servers, or networks that need to be evaluated. Organizations must determine the scope of the scan and the type of vulnerabilities they want to detect.

In addition, it is important to ensure that proper authorization and access permissions are available before scanning the target systems. Unauthorized scanning may disrupt services or violate security policies.

2. Configuration

Once the preparation is complete, the scan must be configured. In this stage, a scan policy is created to define the parameters and behavior of the scan. The policy determines what types of vulnerabilities should be checked, the scanning techniques to be used, and other configuration settings.

After creating the policy, the target assets are defined. These targets may include specific IP addresses, domain names, hostnames, or entire network ranges that need to be analyzed.

3. Scanning

After configuration, the scanning process is initiated. Nessus uses its database of plugins and vulnerability checks to analyze the specified targets. It systematically scans the systems to identify open ports, running services, software versions, and potential vulnerabilities.

The scan may take some time depending on the size of the network and the complexity of the scan configuration.

4. Vulnerability Identification

During the scanning process, Nessus performs a comprehensive security assessment. It identifies vulnerabilities such as missing security patches, insecure configurations, outdated software, and exposed services.

The detected vulnerabilities are categorized based on severity levels such as critical, high, medium, low, or informational. This classification helps security teams understand which issues need immediate attention.

5. Reporting

Once the scan is completed, Nessus generates detailed reports describing the vulnerabilities discovered during the assessment. These reports provide important information including the vulnerability description, severity level, affected systems, and recommended remediation steps.

Reports can be customized and exported in different formats such as PDF, HTML, or CSV, making them useful for documentation, analysis, and sharing with security teams.

6. Remediation

After reviewing the scan results, IT and security teams analyze the vulnerabilities and prioritize them based on their severity and potential impact.

Remediation actions may include installing software patches, updating applications, modifying system configurations, or implementing additional security controls to reduce the risk.

7. Follow-Up and Continuous Monitoring

Security is an ongoing process. Organizations should conduct regular Nessus scans to continuously monitor their systems and networks. Periodic scanning helps identify newly discovered vulnerabilities and ensures that previously detected issues have been properly resolved.

Nessus can also be integrated with other security tools and automation systems to streamline vulnerability management and improve the overall security workflow.

8. Compliance Auditing

In some cases, Nessus scans are used to verify compliance with industry standards and regulations. If compliance checks are enabled in the scan policy, the results will indicate whether the systems meet specific security requirements such as PCI-DSS or CIS benchmarks.

9. Documentation

Maintaining proper documentation is an essential part of vulnerability management. Organizations should keep records of scan results, remediation activities, and compliance reports. These records are useful for audits, security reviews, and tracking improvement over time.

10. User Training

Finally, IT and security teams should be properly trained in using Nessus and interpreting scan results. Proper training ensures that vulnerabilities are accurately identified and effectively addressed.

Understanding Nessus

Nessus is a widely used vulnerability scanning tool that helps organizations identify security weaknesses in their systems and networks. Security professionals use it to evaluate the overall security posture of their infrastructure and detect vulnerabilities before attackers can exploit them.

Key Features of Nessus

Vulnerability Assessment

Nessus scans target systems to detect security weaknesses such as missing patches, weak configurations, outdated software, and other exploitable issues.

Network Scanning

The tool can scan various network devices including servers, routers, switches, firewalls, and workstations. It analyzes open ports, running services, and installed applications.

Plugin-Based Architecture

Nessus operates using a large collection of plugins that check for specific vulnerabilities. These plugins are regularly updated to detect newly discovered security threats.

Compliance Auditing

Nessus can evaluate systems against compliance standards and security frameworks such as PCI-DSS, CIS benchmarks, and other regulatory requirements.

Customization

Users can customize scan policies based on organizational requirements by selecting specific vulnerability checks, defining scan settings, and specifying target assets.

Detailed Reporting

Nessus produces comprehensive reports that highlight vulnerabilities, severity levels, affected systems, and recommended solutions. These reports help security teams prioritize remediation.

Regular Scanning

Organizations often perform regular scans to continuously monitor security and stay protected against evolving threats.

Integration

Nessus can integrate with other security platforms and management systems to automate vulnerability scanning and improve incident response.

Licensing Options

Nessus is available in both free and commercial versions. The free version has limited features, while the commercial version provides advanced capabilities and support.

User-Friendly Interface

The tool offers a web-based interface that is easy to use, making it accessible to both experienced security professionals and beginners.

S No	Vulnerability name	Severity	Plugins
1	Traceroute Information	Info	10287
2	Service Detection	Info	22964
3	OS Identification	Info	11936
4	DNS Server Detection	Info	11011
5	ICMP Timestamp Request Remote Date Disclosure	Info	10114
6	TCP/IP Timestamps Supported	Info	10114
7	Path MTU Discovery	Info	10282
8	Target Credential Status by Authentication Protocol	Info	104410
9	Nessus Scan Information	Info	19506
10	Host Fully Qualified Domain Name	Info	12053

Target Website: vit.ac.in

Target IP Address: 122.184.65.65.22

Report:

1. VULNERABILITY NAME: Traceroute Information

SEVERITY: Info
PLUGIN: Traceroute Information
PORT: N/A

DESCRIPTION:

This plugin performs a traceroute to the remote host and identifies the network path between the scanning system and the target system. It shows the intermediate routers or hops through which the packets travel to reach the target host. This information helps administrators understand the network topology.

SOLUTION:

• Configure network devices to restrict unnecessary ICMP responses.
• Use firewall rules to limit traceroute and network discovery requests from unauthorized sources.

BUSINESS IMPACT:

Although this vulnerability is informational and does not directly compromise the system, attackers may use traceroute results to map the network infrastructure. Knowledge of network paths and intermediate devices may help attackers plan targeted attacks against specific network components.

2. VULNERABILITY NAME: Service Detection

SEVERITY: Info
PLUGIN: Service Detection
PORT: N/A

DESCRIPTION:

The Nessus scanner identifies the services running on open ports of the target system. It determines which services are active and what protocols are being used. This information is useful for system administrators to understand the exposed services on the system.

SOLUTION:

• Disable unnecessary services running on the system.
• Restrict access to services using firewalls or access control mechanisms.

BUSINESS IMPACT:

Information about running services can help attackers identify potential entry points into the system. If vulnerable services are exposed, attackers may attempt to exploit them to gain unauthorized access.

3. VULNERABILITY NAME: OS Identification

SEVERITY: Info
PLUGIN: OS Identification
PORT: N/A

DESCRIPTION:

This plugin attempts to determine the operating system of the target host using network fingerprinting techniques. Identifying the operating system helps security teams understand the environment in which the application is running.

SOLUTION:

• Configure systems to limit OS fingerprinting through firewall rules.
• Disable unnecessary responses that reveal system details.

BUSINESS IMPACT:

Operating system information may help attackers identify known vulnerabilities associated with that OS version. Attackers may use this information to launch targeted exploits against the system.

4. VULNERABILITY NAME: DNS Server Detection

SEVERITY: Info
PLUGIN: DNS Server Detection
PORT: N/A

DESCRIPTION:

The scanner detects whether a Domain Name System (DNS) service is running on the target host. It identifies DNS-related configurations and provides information about the DNS server availability.

SOLUTION:

• Restrict DNS services to authorized users only.
• Use firewall rules to prevent external access to DNS servers when not required.

BUSINESS IMPACT:

Exposed DNS services may allow attackers to gather information about the network infrastructure. Improper DNS configuration may also lead to DNS-based attacks.

5. VULNERABILITY NAME: ICMP Timestamp Request Remote Date Disclosure

SEVERITY: Info
PLUGIN: ICMP Timestamp Request Remote Date Disclosure
PORT: N/A

DESCRIPTION:
This vulnerability occurs when the target system responds to ICMP timestamp requests. These responses reveal the system's current date and time, which can provide useful information to attackers.

SOLUTION:
• Disable ICMP timestamp responses on the system.
• Configure firewall rules to block unnecessary ICMP requests.

BUSINESS IMPACT:
Although this issue does not directly compromise the system, attackers may use timestamp information to synchronize attacks or gather system details that could aid in further exploitation.

6. VULNERABILITY NAME: TCP/IP Timestamps Supported

SEVERITY: Info
PLUGIN: TCP/IP Timestamps Supported
PORT: N/A

DESCRIPTION:
The target system supports TCP timestamps. These timestamps are used to improve network performance and reliability, but they may also reveal system uptime and other timing information.

SOLUTION:
• Disable TCP timestamps if they are not required.
• Configure network settings to limit the exposure of timing information.

BUSINESS IMPACT:
Attackers may use TCP timestamp information to determine system uptime or predict sequence numbers, which could assist in certain types of network attacks.

7. VULNERABILITY NAME: Path MTU Discovery

SEVERITY: Info
PLUGIN: Path MTU Discovery
PORT: N/A

DESCRIPTION:
The scanner identifies the Maximum Transmission Unit (MTU) size supported by the network path between the scanner and the target host. This information helps optimize network packet transmission.

SOLUTION:

• Configure firewalls to control unnecessary network discovery traffic.
• Limit exposure of network configuration details.

BUSINESS IMPACT:

While this is mainly informational, attackers may use network configuration details to better understand the infrastructure and design network-based attacks.

8. VULNERABILITY NAME: Target Credential Status by Authentication Protocol

SEVERITY: Info
PLUGIN: Target Credential Status by Authentication Protocol
PORT: N/A

DESCRIPTION:

This plugin checks whether authentication credentials were successfully used during the scan. It reports the status of credentials used to access the target system.

SOLUTION:

• Ensure proper credential management during vulnerability scans.
• Protect credentials using strong authentication mechanisms.

BUSINESS IMPACT:

Credential-related information can reveal how systems are accessed or authenticated. Improper credential handling may expose sensitive authentication details.

9. VULNERABILITY NAME: Nessus Scan Information

SEVERITY: Info
PLUGIN: Nessus Scan Information
PORT: N/A

DESCRIPTION:

This plugin provides general information about the scan itself, including the scanning tool, scan type, and time of execution. It helps document the scanning process.

SOLUTION:

• No action required since this is informational.
• Maintain secure access to scan reports.

BUSINESS IMPACT:

This plugin does not represent a vulnerability but provides useful documentation for auditing and vulnerability management.

10. VULNERABILITY NAME: Host Fully Qualified Domain Name (FQDN) Resolution

SEVERITY: Info
PLUGIN: FQDN Resolution
PORT: N/A

DESCRIPTION:

The scanner resolves the Fully Qualified Domain Name (FQDN) of the target host. It identifies the domain name associated with the IP address.

SOLUTION:

• Restrict unnecessary DNS information exposure.
• Use secure DNS configurations.

BUSINESS IMPACT:

Domain name information may help attackers identify internal systems or network structures, which could assist them in planning further attacks.

Stage – 3

Title

Design and Implementation of Context-Aware Access Control for Improving Web Application Security

Overview

Context-aware access control is an advanced security approach that enhances traditional access control mechanisms by considering multiple contextual factors before granting access to a system or application. Unlike conventional authentication systems that rely only on usernames and passwords, context-aware access control evaluates several attributes such as user identity, device condition, location, and application risk level to determine whether access should be allowed.

In modern web environments where applications are accessed from different devices and locations, relying solely on basic authentication mechanisms can lead to serious security risks. Attackers may exploit vulnerabilities such as cross-site scripting, SQL injection, and weak authentication mechanisms to gain unauthorized access to sensitive systems. Therefore, implementing context-aware access control helps strengthen the security posture of web applications by ensuring that only legitimate and trusted users are granted access.

The design of context-aware access control begins with verifying the **user identity**. User authentication is the first step in determining whether a user is authorized to access the system. Authentication methods such as strong passwords, multi-factor authentication (MFA), and role-based access control can be implemented to ensure that only legitimate users gain access to sensitive resources. By associating access permissions with specific user roles, organizations can enforce the principle of least privilege, ensuring that users only have access to the resources necessary for their tasks.

The second important factor is the **device posture**. Before granting access to an application, the system evaluates the security condition of the device used by the user. Devices that do not meet security requirements—such as outdated operating systems, disabled firewalls, or missing antivirus protection—may be restricted from accessing critical systems. Ensuring that devices meet security standards reduces the possibility of compromised devices being used to attack web applications.

Another significant factor in context-aware access control is **location awareness**. The system evaluates the geographical location or network environment from which the access request originates. Access attempts from trusted networks or approved locations may be allowed, while connections from suspicious or unknown locations may trigger additional verification steps. This helps detect abnormal behavior and prevents unauthorized access attempts.

The **application risk level** is also considered when designing security policies. Not all applications have the same sensitivity level. For example, a public information portal may require minimal restrictions, whereas financial systems, student databases, or administrative applications require stricter security policies. By classifying applications based on their risk level, organizations can apply stronger authentication and monitoring mechanisms to critical systems.

During the implementation phase, these contextual factors are combined to create dynamic access policies. For example, a policy may allow access to a web application only when the user identity is verified, the device meets security requirements, and the connection originates from an approved location. If any of these conditions are not satisfied, the system may deny access or request additional verification.

By integrating these multiple security layers, context-aware access control significantly reduces the risk of unauthorized access and strengthens the overall security of web applications. This approach ensures that security decisions are based on real-time conditions rather than static rules, making it an effective strategy for protecting modern web environments.

Conclusion

Stage 1

Web application testing plays a vital role in ensuring the security, functionality, and reliability of web-based systems. It involves identifying vulnerabilities such as cross-site scripting, SQL injection, broken authentication, and access control weaknesses. By detecting these issues early, developers can implement appropriate security measures to protect sensitive data and maintain system integrity. Effective web application testing ultimately improves the user experience and reduces the risk of cyberattacks.

Stage 2

The vulnerability assessment conducted using the **Nessus scanner** provided valuable insights into the security status of the target system. The scan identified several informational vulnerabilities related to network configuration, service detection, and system information exposure. Although these findings were not critical threats, they reveal useful information that attackers could potentially exploit. The report highlights the importance of regularly scanning systems, applying security patches, and following best security practices to maintain a secure network environment.

Stage 3

The implementation of context-aware access control enhances the overall security framework of web applications by incorporating multiple contextual factors into the access decision process. By evaluating user identity, device condition, location, and application sensitivity, organizations can ensure that only legitimate and trusted users are granted access to critical

systems. This approach strengthens security defenses, minimizes unauthorized access risks, and supports a more adaptive and intelligent security model suitable for modern web environments.

Future Scope

Stage 1: Future Scope of Web Application Testing

The future of web application testing is rapidly evolving due to the increasing reliance on online services and digital platforms. As organizations continue to develop complex web applications, the need for effective testing strategies will grow significantly.

• **Increased Demand:** The expansion of online services and digital platforms will increase the need for secure and reliable web applications. As a result, web application testing will remain an essential part of the development process.

• **Advanced Technologies:** Emerging technologies such as artificial intelligence, Internet of Things (IoT), and blockchain will introduce new challenges and opportunities in web application testing.

• **Enhanced Security Testing:** With the rise in cyber threats and data breaches, security testing will become more critical. Testers will focus on identifying vulnerabilities and ensuring compliance with security standards.

• **Mobile and Cross-Browser Testing:** As users access web applications from multiple devices and browsers, ensuring compatibility and responsiveness across platforms will be essential.

• **Performance Optimization:** Performance testing will continue to play an important role in ensuring that web applications can handle large numbers of users without performance degradation.

Stage 2: Future Scope of Vulnerability Assessment

The use of vulnerability scanning tools such as Nessus will continue to expand as organizations seek proactive methods to identify and mitigate security risks.

• **AI-Based Vulnerability Detection:** Artificial intelligence and machine learning will enhance vulnerability detection by automatically identifying security weaknesses in complex systems.

• **Integration with DevSecOps:** Vulnerability scanning tools will increasingly integrate with development pipelines to detect security issues during the software development lifecycle.

• **Cloud Security Testing:** As organizations move their applications to cloud platforms, vulnerability assessment tools will focus on securing cloud environments and cloud-based applications.

• **Container and Microservice Security:** With the widespread use of container technologies such as Docker and Kubernetes, vulnerability scanners will play a critical role in securing containerized environments.

Stage 3: Future Scope of Context-Aware Access Control

Context-aware access control systems will continue to evolve as organizations adopt more advanced cybersecurity strategies.

• **AI-Based Risk Analysis:** Machine learning algorithms can analyze user behavior patterns to detect suspicious activities and automatically adjust access permissions.

• **Adaptive Authentication:** Systems may dynamically adjust authentication requirements based on the risk level of the access request.

• **Integration with Threat Intelligence:** Access control systems can integrate with threat intelligence platforms to block access from known malicious sources.

• **Automated Security Policies:** Future systems may automatically update access control policies based on real-time security data.

Topics Explored

1. Web Application Security
2. Vulnerability Assessment
3. Network Security
4. Access Control Mechanisms
5. Threat Intelligence
6. Incident Response
7. Security Operations Center (SOC) Concepts
8. College Network Security Analysis

Tools Explored

• Nessus Vulnerability Scanner